

Estructuras de Control – III

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo



Estructuras de Control III

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Objetivo General

Comprender la relevancia de las estructuras de control en la programación y cómo influyen en el flujo de ejecución de un programa.

Objetivos Específicos

- Identificar los tres tipos principales de estructuras de control: condicionales, repetitivas y de salto, y sus aplicaciones en la resolución de problemas.
- Explicar el uso y funcionamiento de las estructuras condicionales y repetitivas para tomar decisiones y ejecutar tareas repetitivas de manera eficiente.

De Salto continue

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      for (int i = 1; i <= 5; i++) {
6          if (i == 3) {
7              continue; // Si el número es 3, lo salta
8          }
9          cout << "Número: " << i << endl;
10     }
11
12     return 0;
13 }
```

Output

Número: 1

Número: 2

Número: 4

Número: 5

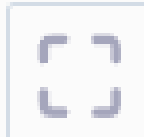
 ¿Qué hace?

- Recorre los números del 1 al 5.
- Cuando llega al número 3, lo salta y no lo imprime

Ejemplos prácticos

De Salto Go to

main.cpp



Share

Run

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      cout << "Inicio del programa" << endl;
6
7      goto saltar; // Salta la línea siguiente
8
9      cout << "Esto no se imprimirá" << endl;
10
11  saltar:
12      cout << "Saltaste una línea con goto" << endl;
13
14      return 0;
15  }
```

Output

```
Inicio del programa
Saltaste una línea con goto
```



¿Qué hace?

- Imprime "Inicio del programa".
- Usa `goto` para **saltar una línea de código**.
- Luego imprime "Saltaste una línea con goto"

Ejemplos prácticos

De Salto Go to

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int i = 1;
6
7     inicio: // Etiqueta a la que se puede saltar
8     if (i <= 5) {
9         cout << "Número: " << i << endl;
10        i++;
11        if (i == 4) {
12            cout << "Salto a la etiqueta inicio (omitimos el 4)" <<
                endl;
13            i++; // Saltamos el número 4
14            goto inicio; // Vuelve a la etiqueta 'inicio'
15        }
16        goto inicio;
17    }
18
19    return 0;
20 }
```

Output

```
Número: 1
Número: 2
Número: 3
Salto a la etiqueta inicio (omitimos el 4)
Número: 5
```



¿Qué hace?

- Usa **goto** para **volver a una etiqueta** llamada inicio.
- **Omite el número 4** a propósito al incrementar i y saltar directamente.

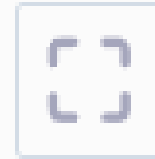
⚠ Nota importante:

- **continue** se usa comúnmente y es **buena práctica** en estructuras repetitivas.
- **goto** es **desaconsejado en programación moderna** porque puede dificultar la lectura y mantenimiento del código. Solo se recomienda en situaciones muy específicas.

Ejemplos prácticos

De Salto Break

main.cpp



Share

Run

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      for (int i = 1; i <= 10; i++) {
6          if (i == 5) {
7              break; // Cuando i es igual a 5, se detiene el ciclo
8          }
9          cout << "Número: " << i << endl;
10     }
11
12     return 0;
13 }
```

Output

Número: 1

Número: 2

Número: 3

Número: 4



¿Qué hace este programa?

- Imprime los números del 1 al 10 usando un ciclo for.
- Pero al llegar al número 5, **usa break para salir del ciclo antes de tiempo.**
- Todo lo que está después del break dentro del ciclo **no se ejecuta m**

Ejemplos prácticos

De Salto Return

¿Qué significa el número que se devuelve con return?

- **return 0;** → indica que el programa **terminó correctamente** (éxito).
- **return 1;**, **return -1;**, u otro valor diferente de 0 → indica que el programa **terminó con un error o alguna condición especial**.

Esto se conoce como **código de salida** y es útil cuando:

- Un programa es llamado por otro (por ejemplo, en sistemas operativos o scripts).
- Se quiere comunicar si **todo salió bien** o si hubo **problemas**

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      cout << "Inicio del programa" << endl;
6
7      return 0; // Termina el programa aquí
8
9      // Esta línea no se ejecuta porque el return anterior finaliza el
       programa
10     cout << "Esto nunca se mostrará" << endl;
11 }
```

Output

Inicio del programa

 **Conclusión:** Sí puedes usar otro número distinto de 0 en return, y es muy útil para indicar errores o estados específicos del programa.

Repaso rapido

Estructuras Condicionales

 Instrucción	 Descripción
<code>if</code>	Ejecuta un bloque si una condición es verdadera.
<code>if...else</code>	Ejecuta un bloque si la condición es verdadera, y otro si es falsa.
<code>else if</code>	Evalúa condiciones adicionales si el <code>if</code> inicial es falso.
<code>switch</code>	Selecciona entre múltiples casos posibles según el valor de una variable.

Estructuras de Salto

 Instrucción	 Descripción
<code>break</code>	Sale de un ciclo o <code>switch</code> .
<code>continue</code>	Salta al inicio de la siguiente iteración del ciclo.
<code>goto</code>	Salta a una etiqueta específica del código. ⚠ Poco recomendable
<code>return</code>	Sale de una función y puede devolver un valor.

Estructuras Repetitivas

 Instrucción	 Descripción
<code>for</code>	Repite un bloque de código un número conocido de veces.
<code>while</code>	Repite mientras se cumpla una condición.
<code>do...while</code>	Repite al menos una vez y luego mientras se cumpla la condición.

"Dominar las estructuras de control es como aprender a dirigir el flujo de tus ideas. Cada instrucción es una decisión que transforma problemas en soluciones."

Estructuras de Control - III

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Estructuras de Control - III

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

